全志 T 系列 hwdisplay2
模块使用说明

版本号: 1.5
发布日期: 2020.12.22

版本历史

版本号	日期	制/修订人	内容描述
0.8	2020.10.20	Tjx	1. 建立版本 2. 添加接口说明
1.0	2020.12.02	Tjx	1. 添加使用简易 demo
1.2	2020.12.17	Tjx	1. 补充部分接口说明 2. 增加接口使用的部分注意事项
1.5	2020.12.22	Tjx	1. 新增 sunxi_display2 相关接口说明

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目 录

1 概述 **1**
1.1 文档简介 1
1.2 相关人员 1

2 模块概述 **2**
2.1 hwdisplay 简介 2
2.2 简要框架 2

3 成员变量 **4**
3.1 图层配置信息 disp_layer_config 4
3.2 图层配置信息 disp_layer_config2 5
3.3 通过 ioctl 获取和设置 layer_config 7
3.3.1 DISP_LAYER_SET_CONFIG 7
3.3.2 DISP_LAYER_GET_CONFIG 8

4 常见接口详情 **9**
4.1 getInstance 9
4.2 hwd_screen1_init 9
4.3 aut_hwd_layer_request 9
4.4 aut_hwd_layer_set_src 10
4.5 aut_hwd_layer_set_rect 11
4.6 aut_hwd_layer_set_zorder 11
4.7 aut_hwd_layer_open 12
4.8 aut_hwd_layer_fb_mode 12
4.9 aut_hwd_layer_set_alpha 13
4.10 aut_hwd_layer_sufacevie 13
4.11 aut_hwd_getLayerConfig 13
4.12 aut_hwd_vsync_enable 14
4.13 getScreenWidth 14
4.14 getScreenHeight 14
4.15 getScreenOutputType 15

5 demo 示例 **16**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 文档简介

此文档为 hwdisplay 的使用说明文档。

1.2 相关人员

使用 hwdisplay 相关接口的开发人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块概述

2.1 hwdisplay 简介

HwDisplay 是 T 系列 linux_sdk 基于 DE 的 IOCTL 接口通过 C++ 封装的一层接口。在摄像头、播放器和显示等模块中都使用了该接口来显示视频数据。

2.2 简要框架

下图是 hwdisplay API 与 DE 显示系统之间的关系。

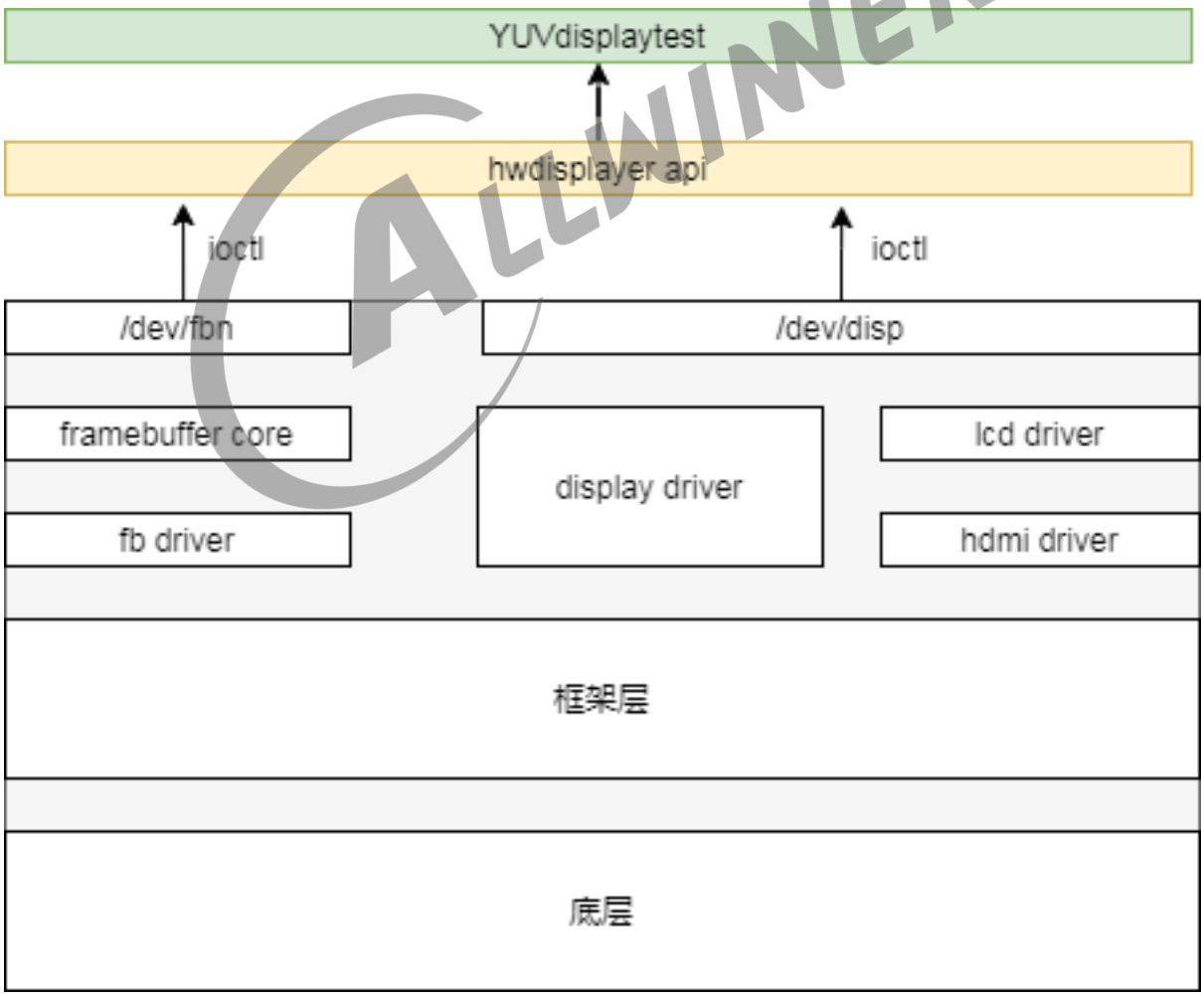

图 2-1: hwdisplay 简要框架

DE 显示系统可划分为三个层面，驱动层，框架层及底层。底层与图形硬件相接，主要负责将上层配置的功能参数转换成硬件所需要的参数，并配置到相应寄存器中。显示框架层对底层进行抽象封装成一个个的功能模块。驱动层对外封装功能接口，通过内核向用户空间提供相应的设备结点及统一的接口。在驱动层，分为四个驱动，分别是 framebuffer 驱动，disp 驱动，lcd 驱动，hdmi 驱动。Framebuffer 驱动与 framebuffer core 对接，实现 linux 标准的 framebuffre 接口。Disp 驱动是是整个显示驱动中的核心驱动模块，所有的接口都由 disp 驱动来提供，包括 lcd 的接口以及 hdmi 的接口。

hwdisplay 通过封装驱动层提供的接口，对应用程序提供相应简便易用的显示操作相关接口。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 成员变量

HWdisplay 是通过对固定的 config 结构体的管理，完成对一系列显示功能调用的分类。主要成员包含有disp_layer_config2 和 layer_stat 前者用来记录各个 screen channal 下 layer 的 config 信息，后者用于记录 layer 的状态。详细介绍如下所示。

```
int ScreenId[LAYERNUM];  //记录特定LAYERNUM的屏幕ID
int ChanelId[LAYERNUM];  //记录特定LAYERNUM的通道ID
int LayerId[LAYERNUM];   //记录特定LAYERNUM的layerID
int Zorder[LAYERNUM];    //记录特定LAYERNUM的Zorder值
struct view_info Surface[LAYERNUM];
unsigned int layerObj[LAYERNUM];
bool CameraDispEnableFlag[LAYERNUM];
static HwDisplay *sHwDisplay;
private:

int mDisp_fd;
int mLayer;
int mScreen;
struct view_info mView;
struct view_info mView1;

static bool     mInitialized;

//static Mutex sLock;
static pthread_mutex_t sLock;
//disp_layer_config2 config;//old use before v1.0.1
disp_layer_config2 layer_cfg[SCREEN_NUM][CHANNEL_NUM][LAYER_NUM];//new 1.1.0 记录各个图层
的config信息
int layer_stat[SCREEN_NUM][CHANNEL_NUM][LAYER_NUM];//new 1.1.0 记录各个图层的状态
```

3.1 图层配置信息 disp_layer_config

- 作用：描述一个图层配置的属性信息
- 成员:
  - info: 图像的信息属性
  - enable: 图层打开的使能标志
  - channel: 图层所在的通道 ID
  - layer_id: 图层 id，此 id 是在通道内的图层 id。
- 结构定义:

```
typedef struct
 {
   disp_layer_info info;
   bool enable;
   unsigned int channel;
   unsigned int layer_id;
 }disp_layer_config;

  typedef struct
  {
    disp_layer_mode mode;
    unsigned char zorder; /*specifies the front-to-back ordering of the layers on the
    screen, the top layer having the highest Z value can't set zorder, but can get */
    unsigned char alpha_mode; //0: pixel alpha; 1: global alpha; 2: global pixel alpha
    unsigned char alpha_value; //global alpha value
    disp_rect screen_win; //display window on the screen
    bool b_trd_out; //3d display
    disp_3d_out_mode out_trd_mode;//3d display mode
    union {
      unsigned int color; //valid when LAYER_MODE_COLOR
      disp_fb_info fb; //framebuffer, valid when LAYER_MODE_BUFFER
    };
    unsigned int id; /* frame id, can get the id of frame
    display currently by DISP_LAYER_GET_FRAME_ID */
  }disp_layer_info;
```

3.2 图层配置信息 disp_layer_config2

- 作用：用于描述一个图层配置的属性信息，与 disp_layer_config 差别在于支持的功能更多，支持 ATW/FBD/HDR 功能。
- 成员：
- format：数据宽度会在 format 体现出来
- atw: 异步时移信息
- eotf: 光电转换特性信息，HDR 图像时需要
- metadata_buf: 指向携带 metadata 的 buffer 的地址
- metadata_size:metadata buffer 的大小
- metadata_flag: 标识 metadata buffer 中携带的信息类型
- 其他: 参考前述
- 结构定义：

```
/\* disp_fb_info2 - image buffer info v2
  \*
  \* @addr: buffer address for each plane
  \* @size: size<width,height> for each buffer, unit:pixels
  \* @align: align for each buffer, unit:bytes
  \* @format: pixel format
  \* @color_space: color space
  \* @trd_right_addr: the right-eye buffer address for each plane,
  \* valid when frame-packing 3d buffer input
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
\* @pre_multiply: indicate the pixel use premultiplied alpha
\* @crop: crop rectangle for buffer to be display
\* @flag: indicate stereo/non-stereo buffer
\* @scan: indicate interleave/progressive scan type, and the scan order
\* @metadata_buf: the phy_address to the buffer contained metadata for
fbc/hdr
\* @metadata_size: the size of metadata buffer, unit:bytes
\* @metadata_flag: the flag to indicate the type of metadata buffer
\* 0 : no metadata
\* 1 << 0: hdr static metadata
\* 1 << 1: hdr dynamic metadata
\* 1 << 4: frame buffer compress(fbc) metadata
\* x : all type could be "or" together
\*/
struct disp_fb_info2 {
  unsigned long long addr[3];
  struct disp_rectsz size[3];
  unsigned int align[3];
  enum disp_pixel_format format;
  enum disp_color_space color_space;
  unsigned int trd_right_addr[3];
  bool pre_multiply;
  struct disp_rect64 crop;
  enum disp_buffer_flags flags;
  enum disp_scan_flags scan;
  enum disp_eotf eotf;
  unsigned long long metadata_buf;
  unsigned int metadata_size;
  unsigned int metadata_flag;
};
/\* disp_layer_info2 - layer info v2
\*
\* @mode: buffer/clolor mode, when in color mode, the layer is widthout buffer
\* @zorder: the zorder of layer, 0~max-layer-number
\* @alpha_mode:
\* 0: pixel alpha;
\* 1: global alpha
\* 2: mixed alpha, compositing width pixel alpha before global alpha
\* @alpha_value: global alpha value, valid when alpha_mode is not pixel alpha
\* @screen_win: the rectangle on the screen for fb to be display
\* @b_trd_out: indicate if 3d display output
\* @out_trd_mode: 3d output mode, valid when b_trd_out is true
\* @color: the color value to be display, valid when layer is in color mode
\* @fb: the framebuffer info related width the layer, valid when in buffer mode
\* @id: frame id, the user could get the frame-id display currently by
\* DISP_LAYER_GET_FRAME_ID ioctl
\* @atw: asynchronous time wrap information
\*/
struct disp_layer_info2 {
  enum disp_layer_mode mode;
  unsigned char zorder;
  unsigned char alpha_mode;
  unsigned char alpha_value;
  struct disp_rect screen_win;
  bool b_trd_out;
  enum disp_3d_out_mode out_trd_mode;
  union {
    unsigned int color;
    struct disp_fb_info2 fb;
  };
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  unsigned int id;
  struct disp_atw_info atw;
};
/\* disp_layer_config2 - layer config v2
\*
\* @info: layer info
\* @enable: indicate to enable/disable the layer
\* @channel: the channel index of the layer, 0~max-channel-number
\* @layer_id: the layer index of the layer widthin its channel

struct disp_layer_config2 {
  struct disp_layer_info2 info;
  bool enable;
  unsigned int channel;
  unsigned int layer_id;
};
```

3.3 通过 ioctl 获取和设置 layer_config

3.3.1 DISP_LAYER_SET_CONFIG

- 作用：该函数用于设置多个图层信息
- 参数：
- handle：显示驱动句柄
- cmd：DISP_SET_LAYER_CONFIG
- arg：arg[0] 为显示通道 0/1；arg[1] 为图层配置参数指针；arg[2] 为需要配置的图层数目
- 返回：
- DIS_SUCCESS: 成功
- 其他: 失败号
- 示例

```
struct
{
  disp_layer_info info,
  bool enable;
  unsigned int channel,
  unsigned int layer_id,
}disp_layer_config;
//设置图层参数，dispfd 为显示驱动句柄
unsigned long arg[3];
struct disp_layer_config config;
unsigned int width = 1280;
unsigned int height = 800;
unsigned int ret = 0;
memset(&config, 0, sizeof(struct disp_layer_config));
config.channnel = 0;//blending channel
config.layer_id = 0;//layer index in the blending channel
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
config.info.enable = 1;
config.info.mode = LAYER_MODE_BUFFER;
config.info.fb.addr[0] = (unsigned long long)mem_in; //FB 地址
config.info.fb.size[0].width = width;
config.info.fb.align[0] = 4;//bytes
config.info.fb.format = DISP_FORMAT_ARGB_8888; //DISP_FORMAT_YUV420_P
config.info.fb.crop.x = 0;
config.info.fb.crop.y = 0;
config.info.fb.crop.width = ((unsigned long)width) << 32;//定点小数。高32bit 为整数，低32bit
  为小数
config.info.fb.crop.height= ((uunsigned long)height)<<32;//定点小数。高32bit 为整数，低32bit
  为小数
config.info.fb.flags = DISP_BF_NORMAL;
config.info.fb.scan = DISP_SCAN_PROGRESSIVE;
config.info.alpha_mode = 2; //global pixel alpha
config.info.alpha_value = 0xff;//global alpha value
config.info.screen_win.x = 0;
config.info.screen_win.y = 0;
config.info.screen_win.width = width;
config.info.screen_win.height= height;
config.info.id = 0;
arg[0] = 0;//screen 0
arg[1] = (unsigned long)&config;
arg[2 = 1; //one layer
ret = ioctl(dispfd, DISP_LAYER_SET_CONFIG, (void*)arg);
```

3.3.2 DISP_LAYER_GET_CONFIG

- 作用：该函数用于获取图层参数
- 参数：
- handle：显示驱动句柄
- cmd：DISP_LAYER_GET_CONFIG
- arg：arg[0] 为显示通道 0/1；arg[1] 为图层配置参数指针；arg[2] 为需要获取配置的图层数目
- 返回：
- DIS_SUCCESS: 成功
- 其他: 失败号
- 示例

```
//设置图层参数，dispfd 为显示驱动句柄
unsigned long arg[3];
struct disp_layer_config config;
memset(&config, 0, sizeof(struct disp_layer_config));
arg[0] = 0; //disp
arg[1] = (unsigned long)&config;
arg[2] = 1; //layer number
ret = ioctl(dispfd, DISP_GET_LAYER_CONFIG, (void*)arg);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 常见接口详情

4.1 getInstance

```
static HwDisplay* getInstance();
```

- 作用：创建 HwDisplay 类
- 参数：无
- 返回：HwDisplay 指针

demo 示例：

```
HwDisplay* gVdecDisp = NULL;
gVdecDisp = HwDisplay::getInstance();
```

4.2 hwd_screen1_init

```
int hwd_screen1_init();
```

- 作用：初始化显示屏幕
- 参数：无
- 返回：
  - 0: 成功
  - -1: 失败

4.3 aut_hwd_layer_request

```
int aut_hwd_layer_request(struct view_info* surface,int screen_id,unsigned int channel,
    unsigned int layer_id);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 作用：申请图层 layer
- 参数：
- surface: 图层位置大小信息
- screen_id：屏幕 id
- channel：通道 id
- layer_id：图层 id
- 返回：
  - 0: 成功
  - -1: 失败

demo 示例：

```
int video_layer0 = -1;
HwDisplay* gVdecDisp = HwDisplay::getInstance();
struct view_info screen = {
            mParam->dispX,
            mParam->dispY,
            ALIGN(mParam->dispW, 32),
            ALIGN(mParam->dispH, 32)
        };

video_layer0 = gVdecDisp->aut_hwd_layer_request(&screen, SCR_ID_DEFAULT, mParam->chLID,
    mParam->lyrID);
```

4.4 aut_hwd_layer_set_src

```
int aut_hwd_layer_set_src(unsigned int hlay, struct src_info *src,unsigned long addr0,int
    share_fd = -1);
```

- 作用：设置源文件信息
- 参数：
- haly: 图层编号
- *src：配置信息，宽、高和模式
- addr0：通道 id
- share_fd：图层 id
- 返回：
  - 0: 成功
  - -1: 失败

demo 示例：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
struct src_info src;

src = {
    ALIGN(src_w, 32),
    ALIGN(src_h, 32),
    DISP_PIXEL_FORMAT_DEFAULT };
ret = gVdecDisp->aut_hwd_layer_set_src(video_layer1, &src, (unsigned long)&addr, share_fd);
```

4.5 aut_hwd_layer_set_rect

```
int aut_hwd_layer_set_rect(unsigned int hlay, struct view_info *src_rect);
```

- 作用：设置图像源显示范围
- 参数：
- haly: 图层编号
- *src_rect: 位置配置信息
- 返回：
  - 0: 成功
  - -1: 失败

demo 示例：

```
struct view_info crop;

crop = { mParam->cropX, mParam->cropY, mParam->cropW, mParam->cropH };

ret = gVdecDisp->aut_hwd_layer_set_rect(video_layer0, &crop);
```

4.6 aut_hwd_layer_set_zorder

```
int aut_hwd_layer_set_zorder(unsigned int hlay,int zorder);
```

- 作用：设置图层叠加数值 zorder
- 参数：
- haly: 图层编号
- zorder:zorder 值
- 返回：
  - 0: 成功

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- -1: 失败

demo 示例:

```
ret = gVdecDisp->aut_hwd_layer_set_zorder(video_layer0, mParam->zorder);
```

4.7 aut_hwd_layer_open

```
int aut_hwd_layer_open(unsigned int hlay);
```

- 作用: 打开图层
- 参数:
- haly: 图层编号
- 返回:
  - 0: 成功
  - -1: 失败

demo 示例:

```
ret = gVdecDisp->aut_hwd_layer_open(video_layer0);
CHECK_DISP_RETURN(ret, "aut_hwd_layer_open");
```

4.8 aut_hwd_layer_fb_mode

```
int aut_hwd_layer_fb_mode(int screen_id,unsigned int type  ,unsigned int mode);
```

- 作用: 设置 fb 模式
- 参数:
- screen_id: 屏幕编号
- type: 类型
- mode: 模式
- 返回:
  - 0: 成功
  - -1: 失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.9 aut_hwd_layer_set_alpha

```
int aut_hwd_layer_set_alpha(unsigned int hlay,int alpha_mode,int alpha_value);
```

- 作用：设置图层透明度
- 参数：
- hlay: 屏幕编号
- alpha_mode: 透明模式
- alpha_value：透明度值
- 返回：
  - 0: 成功
  - -1: 失败

4.10 aut_hwd_layer_sufacevie

```
int aut_hwd_layer_sufaceview(unsigned int hlay, struct view_info* surface)
```

- 作用：设置屏幕上显示窗口的大小
- 参数：
- hlay: 屏幕编号
- surface: 显示窗口参数（x,y,w,h）
- 返回：
- 0: 成功
- -1: 失败

4.11 aut_hwd_getLayerConfig

```
disp_layer_config2* HwDisplay::aut_hwd_getLayerConfig(unsigned int hlay)
```

- 作用：获取指定屏幕的默认 layerconfig 信息
- 参数：
- hlay: 屏幕编号
- 返回：disp_layer_config2 结构体

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.12 aut_hwd_vsync_enable

```
int HwDisplay::aut_hwd_vsync_enable(int screen, int enable)
```

- 作用：开启/关闭 vsync 消息发送功能
- 参数：
- screen: 屏幕编号
- enable: 使能信息
- 返回：

- 0: 成功
- -1: 失败

4.13 getScreenWidth

```
int HwDisplay::getScreenWidth(int screenId)
```

- 作用：获取屏幕宽度
- 参数：
- screen: 屏幕编号
- 返回：屏幕宽度

4.14 getScreenHeight

```
int HwDisplay::getScreenHeight(int screenId)
```

- 作用：获取屏幕高度
- 参数：
- screen: 屏幕编号
- 返回：屏幕高度

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.15 getScreenOutputType

```
int HwDisplay::getScreenOutputType(int screenId)
```

- 作用：获取屏幕输出类型
- 参数：
- screen: 屏幕编号
- 返回：屏幕输出类型

ALLWINNER®

5 demo 示例

demo 描述

该 demo 使用 hwdisplay2 接口在屏幕上显示一个纯色的图层。

```
#include "hwdisp2.h"
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#include<iostream>
using namespace android;

int allocPicMem(paramStruct_t* pops, int size)
{
    int iRet = 0;

    iRet = allocOpen(MEM_TYPE_DMA, pops, NULL);
    if (iRet < 0) {
        printf("ion_alloc_open failed\n");
        return iRet;
    }
    pops->size = size;
    iRet = allocAlloc(MEM_TYPE_DMA, pops, NULL);
    if (iRet < 0) {
        printf("allocAlloc failed\n");
        return iRet;
    }

    return 0;
}

int freePicMem(paramStruct_t* pops)
{
    int iRet = 0;

    allocFree(MEM_TYPE_DMA, pops, NULL);

    return 0;
}
//为图层设置颜色
static void set_color(unsigned int* vir, int width, int height, unsigned int color)
{
    unsigned int* tmp = vir;
    for (int j = 0; j < height; j++) {
        for (int i = 0; i < width; i++) {
            tmp[j * width + i] = color;
        }
```

```
    }
}

int main(int argc, char** argv)
{
    //创建hwdisplay对象
    HwDisplay* mcd = HwDisplay::getInstance();

    paramStruct_t m_DispMemOps;

    //获取当前的屏幕宽和高
    int iSubWidth = mcd->getScreenWidth(0);
    int iSubHeight = mcd->getScreenHeight(0);
    printf("CYDEBUG screen 0 size [%dx%d]\n", iSubWidth, iSubHeight);

    int inPicSize = iSubWidth * iSubHeight * 4;
    //申请地址
    allocPicMem(&m_DispMemOps, inPicSize);
    memset((void*)m_DispMemOps.vir, 0xFF, m_DispMemOps.size);
    set_color((unsigned int*)m_DispMemOps.vir, iSubWidth, iSubHeight, 0xFF0000FF);

    //设置相关的显示参数
    struct view_info frame = { 0, 0, 1280, 800 };
    struct src_info src = { iSubWidth, iSubHeight, DISP_FORMAT_RGBA_8888 };

    int layer0 = mcd->aut_hwd_layer_request(&frame, 0, 0, 0);

    mcd->aut_hwd_layer_set_src(layer0, &src, m_DispMemOps.phy, m_DispMemOps.ion_buffer.
    fd_data.aw_fd);
    //设置图层的zorder值
    mcd->aut_hwd_layer_set_zorder(layer0, 30);
    //显示该图层
    mcd->aut_hwd_layer_open(layer0);
    getchar();

    mcd->aut_hwd_layer_close(layer0);
    freePicMem(&m_DispMemOps);
    return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2020 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。